COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	9/9/2026	**DATE SUBMITTED:**	9/15/2026
NAME:	LOREEN JAE D. PILAPIL	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 8 **SORTING ALGORITHMS PT.2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	I noticed that this creates an array of 100 random numbers.

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "SortingAlgo.h"

const int maxSize=100;
int unsortedArr[maxSize];

void displayArr(int arr[]);

int main()
{
    srand(time(0));

    for (int i =0; i <maxSize; i++)
    {
        unsortedArr[i]=rand() % 100;
    }
    std::cout <<"Unsorted: \n";
    displayArr(unsortedArr);

    return 0;
}

void displayArr(int arr[])
{
    for (int i =0; i< maxSize; i++)
    {
        std::cout <<arr[i]<< " ";
    }

    std::cout <<std::endl;
}
```

```
Unsorted:
22 46 70 25 57 93 0 41 15 41 58 99 83 66 7 6 5 9 26 78 13 38 6 62 7 39 91 53
 38 78 55 93 3 68 15 65 24 81 0 21 76 18 49 90 64 62 81 12 87 40 9 46 58 13
84 89 74 77 2 65 54 15 42 63 92 73 58 96 80 88 74 72 55 20 65 67 89 32 25 36
 17 30 10 15 39 64 45 24 32 1 77 36 1 53 94 59 22 22 80 43
```

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H


template <typename T>
void shellSort(T arr[], int size){
    for (int interval = size/2; interval >= 1; interval/=2){

        for (int i=interval; i < size; i++){
            T temp= arr[i];
            int j= i;

            while (j >= interval && arr[j - interval] > temp){
                arr[j] = arr[j - interval];
                j -= interval;
            }

            arr[j]= temp;
        }
    }
}

#endif
```

```
int main()
{

    srand(time(0));

    for (int i =0; i <maxSize; i++)
    {
        unsortedArr[i]=rand() % 100;
    }
    std::cout <<"Unsorted: \n";
    displayArr(unsortedArr);

    std::cout <<"\nShell sort: \n";
    shellSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    return 0;
}
```

```
Shell sort:
1 1 1 2 3 3 4 4 4 6 7 9 9 9 10 11 11 12 12 13 13 13 14 15 17 19 20 20 20 21
21 21 23 23 24 24 24 27 28 28 31 32 34 34 35 35 37 38 38 39 44 44 44 44 45 4
7 48 49 50 51 52 53 55 56 57 61 61 62 65 66 66 66 68 68 69 69 69 69 70 71 73
 73 73 73 76 76 77 78 78 79 81 82 82 88 88 91 92 94 96 98
```

Observations	I noticed that this sorts an array of 100 random numbers using the shell sorting technique. It arranges the numbers from smallest to largest by comparing elements that are a certain gap apart, gradually reducing the gap until the array is fully sorted.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
template <typename T>
void merge(T arr[], int left, int middle, int right){
    int size1= middle - left + 1;
    int size2= right - middle;

    T* leftArr= new T[size1];
    T* rightArr= new T[size2];

    for (int i=0; i < size1; i++){
        leftArr[i]=arr[left + i];
    }

    for (int j= 0; j < size2; j++){
        rightArr[j] =arr[middle + 1 + j];
    }
    int i= 0;
    int j= 0;
    int k= left;

    while (i < size1 && j < size2){
        if (leftArr[i] <= rightArr[j]){
            arr[k] = leftArr[i];
            i++;
        }
        else{
            arr[k]= rightArr[j];
            j++;
        }
        k++;}

    while (i < size1){
        arr[k]= leftArr[i];
        i++;
        k++;
    }

    while (j <size2){
        arr[k]= rightArr[j];
        j++;
        k++;
    }
```

```
    while (j <size2){
        arr[k]= rightArr[j];
        j++;
        k++;
    }
    delete[] leftArr;
    delete[] rightArr;
}

template <typename T>
void mergeSort(T arr[], int left, int right){
    if (left < right){
        int middle=left +(right - left)/2;

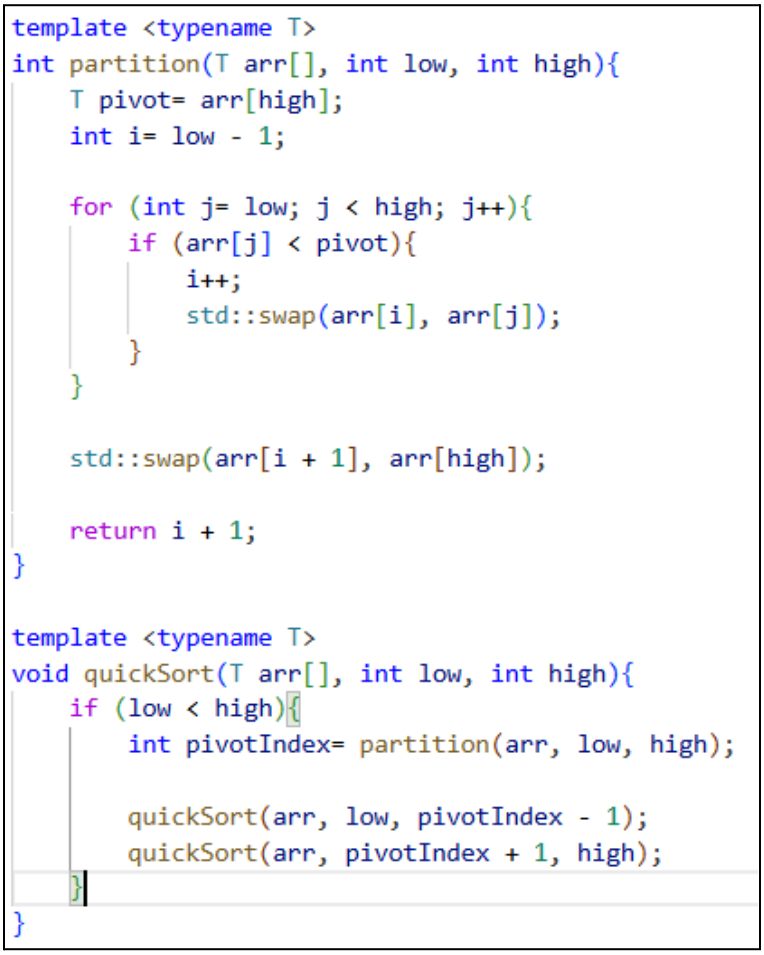
        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);
        merge(arr, left, middle, right);
    }
}
```

```
std::cout <<"\nMerge sort: \n";
mergeSort(unsortedArr, 0, maxSize - 1);
displayArr(unsortedArr);
```

```
Merge sort:
0 0 1 1 3 3 3 5 6 6 6 7 8 9 10 17 19 20 20 21 21 22 25 26 26 27 27 29 29 30
30 31 31 32 33 33 34 35 36 36 36 37 38 38 38 39 40 41 46 49 49 50 51 51 53 5
4 56 57 57 57 57 60 60 62 62 63 63 64 64 65 65 65 67 69 70 70 75 76 77 79 80
 82 82 83 83 85 85 87 87 88 89 89 90 90 91 93 96 96 96 96
```

Observations	I noticed that this sorts an array of 100 random numbers using the merge sorting technique. It arranges the numbers from smallest to largest by dividing the array into smaller parts, sorting each part, and then merging them back together in the correct order.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	
Observations	I noticed that this sorts an array of 100 random numbers using the quick sorting technique. It arranges the numbers from smallest to largest by choosing a pivot, placing smaller numbers before it and larger numbers after it, then repeating the process until the array is sorted.

```
template <typename T>
int partition(T arr[], int low, int high){
    T pivot= arr[high];
    int i= low - 1;

    for (int j= low; j < high; j++){
        if (arr[j] < pivot){
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low, int high){
    if (low < high){
        int pivotIndex= partition(arr, low, high);

        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

```
std::cout <<"\nQuick sort: \n";
quickSort(unsortedArr, 0, maxSize - 1);
displayArr(unsortedArr);
```

```
Quick sort:
0 0 1 1 2 2 2 3 3 6 7 7 8 8 9 11 12 13 15 16 17 20 22 22 23 24 24 26 26 28 2
9 30 30 31 31 32 32 32 34 35 35 36 37 37 38 38 40 41 41 42 42 44 47 47 47 49
 49 50 50 50 51 52 53 55 56 57 58 58 60 60 62 65 65 66 69 71 71 74 74 75 77
79 79 80 81 82 84 84 84 87 87 87 87 90 90 92 92 94 94 95
```

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Problem 1: Yes, we can use merge sort to sort the left and right sublists after partitioning. Merge sort divides each sublist into smaller parts and merges them in sorted order.

Sample array of 10 prices: [99, 129, 149, 79, 9, 899, 399, 30, 89, 70]

```
#ifndef SUPPLE_H
#define SUPPLE_H

void mergeSort(int arr[], int left, int right){
    if (left >= right) return;

    int mid = (left + right)/ 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    int i =left, j= mid + 1, k = 0;
    int temp[10];

    while (i <= mid && j <= right){
        if (arr[i] < arr[j])
            temp[k++] =arr[i++];
        else
            temp[k++] =arr[j++];
    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int x = 0; x< k; x++)
        arr[left + x] =temp[x];
}
int partition(int arr[], int low, int high){
    int pivot =arr[high];
    int i = low;

    for (int j = low; j < high; j++){
        if (arr[j] < pivot) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            i++;
        }
    }
    int temp = arr[i];
    arr[i] = arr[high];
    arr[high] = temp;
    return i;
}
#endif
```

```cpp
#include <iostream>
#include "supple.h"

int main(){
    int prices[] = {99, 129, 149, 79, 9,
                    899, 399, 30, 89, 70};

    int n = 10;
    int pivot = partition(prices, 0, n - 1);

    mergeSort(prices, 0, pivot - 1);
    mergeSort(prices, pivot + 1, n - 1);

    std::cout <<"Sorted prices: ";

    for (int i = 0; i < n; i++)
        std::cout<< prices[i]<< " ";
    return 0;
}
```

```
Sorted prices: 9 30 70 79 89 99 129 149 399 899
```

Step 1: Partition the array using quick sort
Left sublist: [9, 30]
Pivot: [70]
Right sublist: [99, 129, 149, 79, 899, 399, 89]
Step 2: Sort the left sublist using merge sort.
[9, 30] → [9] [30]
Step 3: Sort the right sublist using merge sort.
[99, 129, 149, 79] [899, 399, 89]
[99, 129] [149, 79] [899, 399] [89]
[99] [129] [149] [79] [899] [399] [89]
Step 4: Merge the individual elements
[99] + [129] → [99, 129]
[149] + [79] → [79, 149]
[899] + [399] → [399, 899]
[89]
Step 5: Merge the sorted pairs
[99, 129] + [79, 149] → [79, 99, 129, 149]
[399, 899] + [89] → [89, 399, 899]
Step 6: Merge the sorted groups
[9, 30] + [70] + [79, 99, 129, 149, 89, 399, 899]
[9, 30, 70, 79, 89, 99, 129, 149, 399, 899]

Problem 2: For the given array, quick sort is a suitable choice because it has an average time complexity of O(N log N). Quick sort selects a pivot and partitions the array into smaller sublists. With balanced partitions, each level processes O(N) elements, and the number of levels is O(log N). Merge sort divides the array into two smaller halves until each sublist contains one element. It then merges the sublists in a sorted order. Each division and merging takes O(N) time, and there are O(log N) levels.

Conclusion:

I believe I performed well in this activity because I was able to use the previous sorting activity as a guide in developing my code. This helped me understand the concepts and apply the appropriate sorting algorithms more effectively. I also gained a better understanding of how the algorithms work and how to implement them in C++. At first, I was confused by Problem 1 because I was unsure how to apply another sorting algorithm to the sublists created by quick sort. After reviewing the previous sorting activity, I gained a better understanding of the question and was able to solve it. This experience helped me improve my problem-solving skills and understand how different sorting algorithms can work together.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.